

260330_2차 프로젝트_Unity 3기_우성혁



과제 제출 방법

1. 전체 복사 후 개인페이지에 붙여넣은 뒤 사용합니다.
2. 과제 작성 후 우측 상단에 점3개 아이콘을 클릭한 뒤 [내보내기] > [PDF] 로 노션 페이지를 변환합니다.
 - 내보내기 형식: PDF
 - 데이터베이스 포함: 현재 보기
 - 포함할 콘텐츠: 모든 콘텐츠
3. 다운받은 zip 파일 압축 해제 후, PDF 파일 명을 `yymmdd_2차 프로젝트_Unity 3기_000` 로 변경합니다.
4. 과제 제출란에 PDF 파일을 제출합니다. (zip 파일이 아닌, **PDF 파일**을 제출해주세요!)

**추후 포트폴리오, 프로젝트 문서에 이 과제들을 활용할 수 있으니 일일회고처럼 꼼꼼히 기록해주세요.*

Scrum

**금일 진행한 스크럼 내용을 간략하게 요약해주세요*

**스크럼을 진행하지 않았다면 공란으로 둡니다.*

Project

**프로젝트 작업에서 본인이 기여한 부분에 대해 작성합니다.*

**코드파일이 있다면 스크린샷이나 노션 코드를 사용하여 덧붙여주세요.*

(예: ERD 이미지, Figma 이미지, 코드 스크린샷, Unity 스크린샷 등)

1. 오늘 진행할 작업

세이브 데이터를 JSON 기반으로 수정하여 슬롯 데이터 중복 이슈 해결

대화 툴 샘플 테이블을 사용하여 엔딩 조건 판정 코드를 재구축

JSON 데이터 읽기 코드 변환

2. 진행중인 작업



데이터 세이브 / 로드 기능을 JSON 기반으로 수정

▼ CurData: 현재 플레이 중인 데이터

```
{
  "ID": 1,
  "savedTime": "2026-03-30 13:23:12",
  "StageLock": [
    {
      "stageID": 0,
      "stageLock": true,
      "CheckpointLock": [
        {"CheckpointID": 0, "cpLock": true}
      ]
    },
    {
      "stageID": 1,
      "stageLock": true,
      "CheckpointLock": [
        {"CheckpointID": 0, "cpLock": true},
        {"CheckpointID": 1, "cpLock": true}
      ]
    },
    {
      "stageID": 2,
```

```

        "stageLock": true,
        "CheckpointLock": [
            {"CheckpointID": 0, "cpLock": true},
            {"CheckpointID": 1, "cpLock": true},
            {"CheckpointID": 2, "cpLock": true}
        ]
    },
    {
        "stageID": 3,
        "stageLock": false,
        "CheckpointLock": [
            {"CheckpointID": 0, "cpLock": false},
            {"CheckpointID": 1, "cpLock": true},
            {"CheckpointID": 2, "cpLock": true},
            {"CheckpointID": 3, "cpLock": true}
        ]
    }
],
"MemoryPoint": 100,
"MemoryTag": [
    {"TagName": "self_voice", "tagGet": false}
]
}

```

▼ SaveManager: 데이터 저장 스크립트

```

using System;
using System.IO;
using UnityEngine;
public class SaveManager : MonoBehaviour
{
    public static SaveManager instance;
    public SaveDataObj curData;
    private void Awake()
    {
        if (instance == null) instance = this;
        else Destroy(gameObject);
        curData = ReadCurJSON();
    }
}

```

```

public string GetSavePath(int slot)
{
    return Path.Combine(Application.persistentDataPath, $"SaveSlot{slot}.json");
}

public void CreateSaveData(int slotNumber)
{
    SaveDataObj newData = new SaveDataObj();
    newData.ID = (byte)slotNumber;
    newData.savedTime = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
    newData.MemoryPoint = curData.MemoryPoint;
    newData.StageLock = curData.StageLock;
    newData.MemoryTag = curData.MemoryTag;
    string json = JsonUtility.ToJson(newData, true);
    File.WriteAllText(GetSavePath(slotNumber), json); //선택한 슬롯에 세이브 데이터를 저장
    File.WriteAllText(Path.Combine(Application.persistentDataPath, $"CurData.json"), json); //현재 데이터를 저장한 데이터로 갱신
    SaveUIManager.instance.savePopup.CloseSavePopup();
    StartCoroutine(SaveUIManager.instance.popup_saved.SaveAlarm());
}

public SaveDataObj LoadSaveData(int slotNumber)
{
    string path = GetSavePath(slotNumber);
    if (!File.Exists(path))
    {
        Debug.LogWarning($"세이브 파일 없음: {path}");
        return null;
    }
    string json = File.ReadAllText(path);
    SaveDataObj data = JsonUtility.FromJson<SaveDataObj>(json);
    return data;
}

```

```

    public static SaveDataObj ReadCurJSON()
    {
        SaveDataObj newData = new SaveDataObj();
        string path = Path.Combine(Application.persistentDataPath, $"CurData.json");
        string jsonFile = File.ReadAllText(path);
        newData = JsonUtility.FromJson<SaveDataObj>(jsonFile);
        return newData;
    }
}

```

▼ LoadSlotManager: 세이브 데이터 슬롯 동작 스크립트

```

using System.IO;
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
//버튼 터치 시 세이브 데이터를 로드하여 게임 시작
public class LoadSlotManager : MonoBehaviour
{
    public TMP_Text slotNum;
    public Text slotTime;
    public Text slotCheck;
    public int slotIdx;
    public SaveDataObj slotData;
    void OnEnable()
    {
        if (SaveManager.instance == null)
        {
            Debug.LogError("SaveManager.instance가 아직 초기화되지 않았습니다.");
            return;
        }
        slotData = SaveManager.instance.LoadSaveData(slotIdx);
        if (slotData == null)
        {

```

```

        slotNum.text = slotIdx.ToString();
        slotTime.text = "No Save Data";
        slotCheck.text = "-";
        return;
    }
    slotNum.text = slotData.ID.ToString();
    slotTime.text = slotData.savedTime;
    LastCheckpoint();
}
public void LastCheckpoint()
{
    int lastStage = -1;
    int lastCheckpoint = -1;
    for (int i = 0; i < slotData.StageLock.Count; i++)
    {
        if (!slotData.StageLock[i].stageLock) lastStage = i;
        for (int j = 0; j < slotData.StageLock[i].CheckpointLock.Count; j++)
        {
            if (!slotData.StageLock[i].CheckpointLock[j].cpLock) lastCheckpoint = j;
        }
    }
    slotCheck.text = $"Stage {lastStage} - {lastCheckpoint}";
}
public void SaveGame(int slotID)
{
    SaveManager.instance.CreateSaveData(slotID);
}
public void LoadGame()
{
    slotData = SaveManager.instance.LoadSaveData(slotIdx);
    if (slotData == null)
    {

```

```

        Debug.LogError("로드할 세이브 데이터가 없습니
다.");
        return;
    }
    SaveManager.instance.curData = slotData;
    File.WriteAllText(Path.Combine(Application.persi
stentDataPath, $"CurData.json"), JsonUtility.ToJson(Save
Manager.instance.curData, true));
    SceneManager.LoadSceneAsync($"LobbyScene");
}
}

```



JSON에 입력한 엔딩 조건 데이터를 게임 엔딩 분기에 적용

▼ EndingRule: 진엔딩 조건 설정 데이터

```

{
  "EndingRule": [
    {
      "ruleId": "END_TRUE_01",
      "isTrueEnding": true,
      "minReconstructionRate": 80,
      "requiredTags": "shared_childhood;shared_dream;unf
inished_confession;self_voice",
      "requiredFlags": "final_puzzle_cleared",
      "failDowngradeTo": "Normal",
      "outputEndingStateId": "ESP_TRUE_DEFAULT",
      "notes": "진엔딩 기본 규칙"
    }
  ]
}

```

```

    },
    {
        "ruleId": "Normal",
        "isTrueEnding": false,
        "minReconstructionRate": 0,
        "outputEndingStateId": "ESP_NORMAL_DEFAULT",
        "notes": "노멀 엔딩 기본 규칙"
    }
]
}

```

▼ EndingRule_Data: 진엔딩 조건 JSON 변환 코드

```

using System.Collections.Generic;
[System.Serializable]
public class EndingRuleFile //전체 JSON 파일 데이터
{
    public List<EndingRule_Data> EndingRule; //각 줄마다
    //별개의 데이터가 할당되도록
}
[System.Serializable]
public class EndingRule_Data //개별 데이터 줄
{
    public string ruleId;
    public bool isTrueEnding;
    public int minReconstructionRate;
    public string[] requiredTags;
    public string[] requiredFlags;
    public string failDowngradeTo;
    public string outputEndingStatId;
    public string notes;
}

```

▼ EndingManager: 엔딩 조건 계산 및 엔딩 출력 코드

```

using System.Collections;
using System.IO;
using UnityEditor;
using UnityEngine;

```



```

using UnityEngine.UI;
public class EndingManager : MonoBehaviour
{
    private AudioSource BGMSource;
    private bool isCompleteEnding;
    public AudioClip CompleteEndingBGM;
    public AudioClip NormalEndingBGM;
    public AudioClip ThankstoBGM;
    public Image endingImage;
    public GameObject endSkipButton;
    public WaitForSecondsRealtime canSkipWFS;
    public WaitForSecondsRealtime EndingDuration;
    public Image thankstoImage;
    public GameObject AppEndButton;
    private Coroutine endingPlayCoroutine; // 실행 중인
엔딩 코루틴 저장용
    void Awake()
    {
        BGMSource = GetComponent<AudioSource>();
        endingImage.enabled = true;
        thankstoImage.enabled = false;
        endSkipButton.SetActive(false);
        AppEndButton.SetActive(false);
        canSkipWFS = new WaitForSecondsRealtime(5f); //
엔딩 시작 5초 후 스킵 가능
    }
    private void OnEnable() //엔딩 신 활성화 시점에 트루엔딩
판정
    {
        bool ReconstructionRateCond = EndingConditionData.memory_reconstruction_rate >= EndingPoint();
        isCompleteEnding = ReconstructionRateCond && EndingConditionData.self_voice;
    }
    public static int EndingPoint()
    {
        string path = Path.Combine(Application.dataPath,
"JSON/EndingRule.json");

```

```

        if (!File.Exists(path))
        {
            Debug.LogError("EndingRule.json 파일을 찾을 수
없습니다: " + path);
            return 80;
        }
        string json = File.ReadAllText(path);
        EndingRuleFile endingRule = JsonUtility.FromJson
<EndingRuleFile>(json);
        if (endingRule == null || endingRule.EndingRule
== null || endingRule.EndingRule.Count == 0)
        {
            Debug.LogError("EndingRule 데이터 파싱 실패");
            return 80;
        }
        int rate = endingRule.EndingRule[0].minReconstru
ctionRate;
        return rate;
    }
    private IEnumerator Start()
    {
        if (isCompleteEnding)
        {
            CompleteEnding();
        }
        else
        {
            NormalEnding();
        }
        endingPlayCoroutine = StartCoroutine(EndingPlay
()); // EndingPlay 코루틴 시작 및 저장 (스킵 시 중단할 수 있
도록)

        yield return canSkipWFS;
        endSkipButton.SetActive(true);
    }
    void CompleteEnding()
    {
        BGMSource.clip = CompleteEndingBGM;
    }

```

```

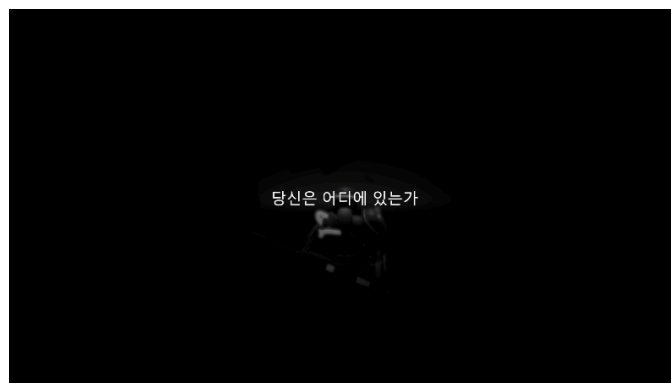
        endingImage.color = new Color (1f,1f,1f,1f);
        EndingDuration = new WaitForSecondsRealtime(20
f); //엔딩 시작 20초 후 종료
    }
    void NormalEnding()
    {
        BGMSource.clip = NormalEndingBGM;
        endingImage.color = new Color(0.8f, 0.8f, 0.8f);
        EndingDuration = new WaitForSecondsRealtime(10
f); //엔딩 시작 10초 후 종료
    }
    IEnumerator EndingPlay()
    {
        BGMSource.Play();
        yield return EndingDuration;
        EndingClear();
    }
    public void OnEndingSkip()
    {
        if (endingPlayCoroutine != null) // 실행 중인 엔
딩 코루틴 중단
        {
            StopCoroutine(endingPlayCoroutine);
            endingPlayCoroutine = null;
        }
        EndingClear();
    }
    private void EndingClear()
    {
        if (endingPlayCoroutine != null) // 이미 실행 중
인 엔딩 플레이 중단
        {
            StopCoroutine(endingPlayCoroutine);
            endingPlayCoroutine = null;
        }
        if (BGMSource != null && BGMSource.isPlaying)
// 엔딩 BGM 중지
        {

```

```

        BGMSource.Stop();
    }
    endingImage.enabled = false;
    endSkipButton.SetActive(false);
    if (isCompleteEnding && ThankstoBGM != null) //
트루엔딩 후 BGM 재생
    {
        BGMSource.clip = ThankstoBGM;
        BGMSource.Play();
    }
    thankstoImage.enabled = true;
    AppEndButton.SetActive(true);
}
public void END()
{
#if UNITY_EDITOR
    EditorApplication.isPlaying = false; //에디터 종
료
#else
    Application.Quit(); //앱 종료
#endif
}
}

```



스테이지 0(새 게임 시작 후 처음 진입하는 오프닝 신) 기능 구현

- 암전 상태에서 Post-processing Volume Profile을 교체하여 블러 전환 효과
- 특정 트리거 도달 시 텍스트 출력 / 다음 신 이동 효과

3. 내일 개발 계획 및 목표

스테이지 0 레벨 제작

Trouble Shooting

**금일 개발 과정 중, 문제를 해결한 경험이 있다면 기술해주세요.*

1. 문제 상황 파악

- a. json으로 입력한 데이터가 반영되지 않는 문제

2. 문제 발생 조건 파악

- a. json 파일에 입력한 변수 이름이 코드에 입력한 변수 이름과 다르면 json의 데이터를 코드로 입력시키지 못하는 이슈

3. 결과 검증, 해결

- a. json 파일에 입력한 텍스트로 코드에 입력한 변수의 이름을 변경하여 해결

그 외 과제 제출 (선택기입)

Github / Blog

**별도로 회고를 정리하는 깃헙, 블로그 게시글 링크가 있다면 첨부해주세요.*

- Github:
- Blog:
- Velog:

기술탐구

**프로젝트 작업을 위해 추가로 학습한 게시글이 있다면 링크를 첨부해주세요*

- (예시) 부트스트랩 설치방법: <https://datamoney.tistory.com/149>
- 링크 1:
- 링크 2: